Software Verification Plan

Projekttitel	Feels Like
Erstellt am	6. Februar 2026
Zuletzt aktualisiert von	Julian Stengele
Zuletzt aktualisiert um	5. März 2026 10:07
Überprüfer/-innen	Julian Stengele Atu Sophie Lazarjan Isabella Schwarz
Erstellt von	Julian Stengele
Kategorie	Strategiedokument

Versionsverwaltung

Version	Datum	Änderungen
V 1.0.0	6. Februar 2026	Initialversion
V 1.0.1	11. Februar 2026	Anpassung des Phasenmodells (7.1)
V 1.1.0	05. März 2026	Umfassende Definition der implementierten Tests

1. Einleitung & Zielsetzung

Dieses Dokument beschreibt die Strategie zur Qualitätssicherung und Verifikation der Software "Feels Like".
Ziel ist es, sicherzustellen, dass die implementierten Funktionen den Anforderungen (aus der Anforderungsanalyse) entsprechen und das System stabil läuft.

2. Anwendungsbereich (Scope)

Verifiziert werden folgende Komponenten des Systems (gemäß SDD):

1. **Backend (Server):**
    - REST-API Endpunkte (Authentifizierung, Athletenverwaltung, Training).
    - Datenbank-Integrität (SQLite/Sequelize Models).
    - Geschäftslogik (z. B. Ranking-Berechnungen).

2. **Frontend (Client):**
    - Benutzeroberfläche (Login, Dashboard, Formulare).
    - Client-seitige Validierung.
    - Routing und Navigation.

3. Verifikationsstrategie

Wir wenden eine Kombination aus statischen und dynamischen Testverfahren an, wie in der Vorlesungseinheit 2.4 beschrieben.

3.1 Konstruktive Qualitätssicherung (Statisch)

- **Code-Richtlinien:** Nutzung von `ESLint` (bereits konfiguriert), um Syntaxfehler und Stilbrüche automatisch zu erkennen.
- **Peer Reviews:** Es gilt das Vier-Augen-Prinzip. Kein Feature-Branch wird in den `dev` Branch gemerged, ohne dass ein anderes Teammitglied den Code auf GitHub gesichtet und freigegeben hat (Pull Request Review).
- **Projektmanagement (GitHub Project Board):** Die Entwicklung wird agil in Sprints organisiert. Aufgaben werden als **GitHub Issues** im **GitHub Project Board** des Repositories erfasst und den Spalten *Backlog* → *Ready* → *In Progress* → *In Review* → *Done* zugeordnet. Jedes Issue ist einem Sprint zugewiesen und wird mit einem **Feature-Branch** verknüpft (z. B. `feature/training-crud`). Kein Feature-Branch wird direkt in `main` gemerged — der Übergang von *In Review* nach *Done* erfolgt erst nach bestandenem Pull Request Review..

3.2 Testmethoden (Dynamisch)

A. Unit Tests

- Testen einzelner Funktionen in Isolation.
- **Fokus:** Komplexe Logik im Backend (z. B. `RankingService`, siehe SDD).
- **Methode:** Äquivalenzklassentests (siehe 3.3).

B. Integrationstests / API-Tests

- Testen der Schnittstellen zwischen Frontend und Backend.
- **Tool:** Postman oder Thunder Client.
- **Szenario:** Login durchführen → Token erhalten → Mit Token geschützte Route aufrufen.

C. Systemtests (Manuell)

- Black-Box-Tests der gesamten Anwendung über die Benutzeroberfläche.
- Prüfung gegen die Use Cases (z. B. "Als Athlet möchte ich ein Training erfassen").

4. Verifikationsumgebung

4.1 Hardware

- Entwickler-Laptops (Windows/Mac/Linux) als lokale Testumgebung.

4.2 Software & Tools

- **IDE:** Visual Studio Code.
- **Laufzeitumgebung:** Node.js (v18+).
- **Datenbank:** SQLite (lokal).
- **API-Testing:** Postman / cURL **(voraussichtlich!)**
- **Linter:** ESLint (statische Analyse).
- **Browser:** Google Chrome / Firefox (für Frontend-Tests).

5. Test-Definitionen

5.1 Implementierte Unit-Tests (Serverseitig)

Test-ID	Testfall	Methode	Typ	Erwartetes Ergebnis
UT-01	Gültige vollständige Daten akzeptieren	`validateTrainingsDaten`	Aquivalenzklasse (gultig)	`valid: true`, keine Fehler
UT-02	Daten ohne optionale Felder akzeptieren	`validateTrainingsDaten`	Aquivalenzklasse (gultig)	`valid: true`
UT-03	distanz = 0 akzeptieren	`validateTrainingsDaten`	Grenzwert	`valid: true`
UT-04	Alle erlaubten Sportarten akzeptieren	`validateTrainingsDaten`	Aquivalenzklasse (gultig)	`valid: true` fur jede Sportart
UT-05	feelsLikeScore Randwert 1 akzeptieren	`validateTrainingsDaten`	Grenzwert (untere Grenze)	`valid: true`
UT-06	feelsLikeScore Randwert 10 akzeptieren	`validateTrainingsDaten`	Grenzwert (obere Grenze)	`valid: true`
UT-07	datum fehlt → Fehler	`validateTrainingsDaten`	Aquivalenzklasse (ungultig)	`valid: false`, Fehler

Test-ID	Testfall	Methode	Typ	Erwartetes Ergebnis
				enthalten "Datum"
UT-08	sportart fehlt → Fehler	`validateTrainingsDaten`	Aquivalenzklasse (ungultig)	`valid: false`
UT-09	dauer fehlt → Fehler	`validateTrainingsDaten`	Aquivalenzklasse (ungultig)	`valid: false`
UT-10	feelsLikeScore fehlt → Fehler	`validateTrainingsDaten`	Aquivalenzklasse (ungultig)	`valid: false`
UT-11	Alle Pflichtfelder fehlen → 4+ Fehler	`validateTrainingsDaten`	Aquivalenzklasse (ungultig)	`valid: false`, >= 4 Fehler
UT-12	Ungultige Sportart → Fehler	`validateTrainingsDaten`	Aquivalenzklasse (ungultig)	`valid: false`
UT-13	dauer = 0 → Fehler	`validateTrainingsDaten`	Grenzwert	`valid: false`
UT-14	Negative dauer → Fehler	`validateTrainingsDaten`	Grenzwert	`valid: false`
UT-15	Negative distanz → Fehler	`validateTrainingsDaten`	Aquivalenzklasse (ungultig)	`valid: false`
UT-16	feelsLikeScore = 0 → Fehler	`validateTrainingsDaten`	Grenzwert (unter Minimum)	`valid: false`
UT-17	feelsLikeScore = 11 → Fehler	`validateTrainingsDaten`	Grenzwert (uber Maximum)	`valid: false`
UT-18	feelsLikeScore Dezimalzahl → Fehler	`validateTrainingsDaten`	Aquivalenzklasse (ungultig)	`valid: false`
UT-19	Training mit gultigen Daten erstellen	`trainingErfassen`	Funktionstest	Mock-Create aufgerufen, Training zuruckgegeben
UT-20	Training mit ungultigen Daten → 400	`trainingErfassen`	Fehlerfall	Error mit Status 400
UT-21	distanz null wenn nicht angegeben	`trainingErfassen`	Spezialfall	`distanz: null` im Create-Aufruf
UT-22	Alle Trainings laden	`alleTrainings`	Funktionstest	findAll mit athletId, sortiert

Test-ID	Testfall	Methode	Typ	Erwartetes Ergebnis
				nach Datum DESC
UT-23	Leeres Array wenn keine Trainings	`alleTrainings`	Grenzfall	Leeres Array
UT-24	Training erfolgreich aktualisieren	`trainingAendern`	Funktionstest	save() aufgerufen, aktualisierte Daten
UT-25	Training nicht gefunden → 404	`trainingAendern`	Fehlerfall	Error mit Status 404
UT-26	Fremdes Training → 403	`trainingAendern`	Autorisierung	Error mit Status 403
UT-27	Ungultige neue Daten → 400	`trainingAendern`	Fehlerfall	Error mit Status 400
UT-28	Training erfolgreich loschen	`trainingLoeschen`	Funktionstest	destroy() aufgerufen
UT-29	Training nicht gefunden → 404	`trainingLoeschen`	Fehlerfall	Error mit Status 404
UT-30	Fremdes Training → 403	`trainingLoeschen`	Autorisierung	Error mit Status 403

Testwerkzeug: Jest 30.2.0 mit Mocking (jest.mock)

Teststrategie: Das Sequelize-Model `Trainingseinheit` wird gemockt (create, findAll, findByPk), sodass die Tests **ohne Datenbank** laufen (isolierte Unit-Tests).

5.2 Testausschlüsse

Noch nicht durch Tests abgedeckt (Stand: 05.03.2026):

- `authController` (Login/Logout) -- kein Unit-Test vorhanden
- `userController` (profilLaden) -- kein Unit-Test vorhanden
- `authMiddleware` (JWT-Verifizierung) -- kein Unit-Test vorhanden
- Frontend-Komponenten -- keine automatisierten Tests
- Integrationstests (API-Endpunkte mit Datenbank)
- E2E-Tests (vollständiger User-Flow)

6. Rollen & Verantwortlichkeiten

Damit die Qualitätssicherung nicht vergessen wird, werden folgende Rollen zugewiesen:

- **Entwickler (Author):** Erstellt den Code und führt lokale Tests durch (Schreibt ggf. Unit-Tests).
- **Reviewer:** Prüft den Pull Request eines anderen Entwicklers auf Logikfehler und Einhaltung der Coding Standards.
- **Tester:** Führt manuelle Systemtests vor einem Release durch.

7. Zeitplan und Meilensteine

Die Verifikation erfolgt begleitend zur Entwicklung ("Continuous Verification"). Da das Projekt agil (in Sprints) durchgeführt wird, passen sich die konkreten Testinhalte den jeweiligen Entwicklungszielen an.

7.1 Phasenmodell der Verifikation

Der Verifikationsprozess folgt dem Fortschritt der Implementierung:

Phase	Zeitraum	Fokus der Verifikation	Methoden
Phase 1: Initialisierung	Projektstart (Sprint 0/1)	Sicherstellung der Entwicklungsfähigkeit. Setup der Linter (ESLint), der Test-Datenbank und der CI/CD-Grundlagen.	Konfiguration, erste Tests
Phase 2: Komponenten-Tests	Fortlaufend	Prüfung neuer Module direkt nach der Erstellung. Ziel ist es, Logikfehler in Services oder Helper-Funktionen früh zu finden.	Unit Tests, Code Reviews
Phase 3: Integrations-Tests	Nach Feature-Implementierung	Prüfung des Zusammenspiels von Client (Frontend) und Server (API). Werden Daten korrekt übertragen und gespeichert?	API-Tests (Postman), Manuelle Tests
Phase 4: System-Tests	Vor Meilenstein-Abgaben	End-to-End Prüfung der gesamten Anwendung aus Nutzersicht. Funktionieren die Abläufe (Workflows) durchgängig?	Manuelle Tests
Phase 5: Abschluss	Projektende	Finaler Review aller Artefakte und Prüfung der Traceability (Wurden alle Anforderungen getestet?).	Audit, Abschlussbericht

7.2 Verifikations-Meilensteine

Die Meilensteine definieren den qualitativen Zustand der Software am Ende eines Zeitabschnitts:

Sprint 1: Infrastruktur & Basis-Funktionalität

- **Ziel:** Das System ist grundlegend lauffähig ("Walking Skeleton").
- **Verifikations-Fokus:**
  - Stabilität der Architektur (Client-Server-Verbindung).
  - Sicherheit der Authentifizierung (Login/Registrierung, Token-Handling).
  - Korrekte Anbindung der Datenbank.
- **Methode:** Code Reviews und exploratives Testen der Basis-Routen.

Sprint 2-3: Erweiterung der Kernfunktionen

- **Ziel:** Die Hauptanwendungsfälle (Use Cases) sind funktional implementiert.
- **Verifikations-Fokus:**
  - Datenintegrität bei CRUD-Operationen (Erstellen, Lesen, Aktualisieren, Löschen von Datensätzen).
  - Validierung von Benutzereingaben (Fehlerbehandlung bei ungültigen Daten).
- **Methode:** Durchführung definierter Integrations-Tests für alle neu hinzugefügten API-Endpunkte.

Sprint 4: Finalisierung & Logik

- **Ziel:** Das System ist vollständig, stabil und bereit für die Abnahme.
- **Verifikations-Fokus:**
  - Prüfung komplexer Geschäftslogiken und Berechnungen.
  - Usability-Tests der Benutzeroberfläche (UX).
  - Grenzfall-Analysen (Edge Cases).
- **Methode:** Vollständiger Systemtest aller Use Cases und Regressionstests (Sicherstellen, dass alte Funktionen noch gehen).

8. Dokumentation & Nachverfolgbarkeit

Um sicherzustellen, dass keine Anforderung vergessen wurde, wird eine einfache **Traceability-Matrix** gepflegt.

- **Verfahren:** Jeder definierte Testfall (TC-ID) wird einer Anforderung (UC-ID aus dem Pflichtenheft) zugeordnet.
- **Fehler-Tracking:** Gefundene Bugs werden als **GitHub Issues** erfasst und mit dem Label `bug` versehen. Im Pull Request wird referenziert, welcher Testfall fehlgeschlagen ist.